

```
words.foldLeft("")( _ + _ ) == ((" " + "Hic") + "Est") + "Index"
words.foldRight("")( _ + _ ) == "Hic" + ("Est" + ("Index" + " "))
```

We can write a general function concatenate that folds a list with a monoid:

```
def concatenate[A](as: List[A], m: Monoid[A]): A =
  as.foldLeft(m.zero)(m.op)
```

But what if our list has an element type that doesn't have a Monoid instance? Well, we can always map over the list to turn it into a type that does:

```
def foldMap[A,B](as: List[A], m: Monoid[B])(f: A => B): B
```



EXERCISE 10.5

Implement foldMap.



EXERCISE 10.6

Hard: The foldMap function can be implemented using either foldLeft or foldRight. But you can also write foldLeft and foldRight using foldMap! Try it.

10.3 Associativity and parallelism

The fact that a monoid's operation is associative means we can choose how we fold a data structure like a list. We've already seen that operations can be associated to the left or right to reduce a list sequentially with foldLeft or foldRight. But if we have a monoid, we can reduce a list using a *balanced fold*, which can be more efficient for some operations and also allows for parallelism.

As an example, suppose we have a sequence a, b, c, d that we'd like to reduce using some monoid. Folding to the right, the combination of a, b, c , and d would look like this:

```
op(a, op(b, op(c, d)))
```

Folding to the left would look like this:

```
op(op(op(a, b), c), d)
```

But a balanced fold looks like this:

```
op(op(a, b), op(c, d))
```

Note that the balanced fold allows for parallelism, because the two inner op calls are independent and can be run simultaneously. But beyond that, the more balanced tree structure can be more efficient in cases where the cost of each op is proportional to

the size of its arguments. For instance, consider the runtime performance of this expression:

```
List("lorem", "ipsum", "dolor", "sit").foldLeft("")(_ + _)
```

At every step of the fold, we're allocating the full intermediate `String` only to discard it and allocate a larger string in the next step. Recall that `String` values are immutable, and that evaluating `a + b` for strings `a` and `b` requires allocating a fresh character array and copying both `a` and `b` into this new array. It takes time proportional to `a.length + b.length`.

Here's a trace of the preceding expression being evaluated:

```
List("lorem", ipsum", "dolor", "sit").foldLeft("")(_ + _)
List("ipsum", "dolor", "sit").foldLeft("lorem")(_ + _)
List("dolor", "sit").foldLeft("loremipsum")(_ + _)
List("sit").foldLeft("loremipsumdolor")(_ + _)
List().foldLeft("loremipsumdolorsit")(_ + _)
"loremipsumdolorsit"
```

Note the intermediate strings being created and then immediately discarded. A more efficient strategy would be to combine the sequence by halves, which we call a *balanced fold*—we first construct `"loremipsum"` and `"dolorsit"`, and then add those together.



EXERCISE 10.7

Implement a `foldMap` for `IndexedSeq`.⁴ Your implementation should use the strategy of splitting the sequence in two, recursively processing each half, and then adding the answers together with the monoid.

```
def foldMapV[A,B](v: IndexedSeq[A], m: Monoid[B])(f: A => B): B
```



EXERCISE 10.8

Hard: Also implement a *parallel* version of `foldMap` using the library we developed in chapter 7. Hint: Implement `par`, a combinator to promote `Monoid[A]` to a `Monoid[Par[A]]`,⁵ and then use this to implement `parFoldMap`.

```
import fpinscala.parallelism.Nonblocking._

def par[A](m: Monoid[A]): Monoid[Par[A]]
def parFoldMap[A,B](v: IndexedSeq[A], m: Monoid[B])(f: A => B): Par[B]
```

⁴ Recall that `IndexedSeq` is the interface for immutable data structures supporting efficient random access. It also has efficient `splitAt` and `length` methods.

⁵ The ability to “lift” a `Monoid` into the `Par` context is something we’ll discuss more generally in chapters 11 and 12.